

Using Logistic Regression and XGBoost for Bot Detection

Elias Schlie

u783766

Tilburg University

e.p.schlie@tilburguniversity.edu

Abstract

The activity of bots on social media platforms is increasingly becoming a problem and platform providers are seeking ways to prevent bots from infiltrating their systems. The following project evaluates two simple machine learning algorithms on their ability to discern between bots and users based on their usage data. The algorithms applied are logistic regression, which was able to reach an accuracy of 97%, and XGBoost, **chenXGBoostScalableTree2016** which reached 98% accuracy.

1 Introduction

With the rise of large language models, bots now have the ability to immitate human behavior better than ever before. This makes discerning them from actual users increasingly hard, and more sophisticated methods are needed to detect them.

2 Data and Preprocessing

2.1 Dataset Description

The "bots_vs_users.csv" dataset contains information on 5874 social media accounts, exactly half of which are bots. Every account has 59 features, that include 14 numerical ones like their `average_views` and 45 categorical ones like the city they live in. 41 of the categorical features are binary like the the variable `has_profile_picture` for example. Most of the features include a large number of missing or unknown values. 4483 accounts don't contain any numerical information, while for the individual categorical features, between 5006 and 24 of the accounts carry the label "unknown". Only the target variable "target", which indicates if the account is a bot or not has no missing features.

2.2 Data split

Before any preprocessing was applied, the data was split into a training and a test set using a stratified 80/20 split. This was done prior to preprocessing due to the reason that some of the preprocess-

ing steps learn from the data and thus would lead to data leakage if applied to the entire dataset at once. Model hyperparameters were validated based on a k-fold cross validation on the training set. To ensure reproducibility, this and every following step that involves randomness was seeded with the seed 42.

2.3 Preprocessing and Feature Engineering

All the preprocessing and feature engineering steps were implemented in a single `DataCleaner` class. The class's `fit_transform` method was used on the training data and transforms it while learning distributional features from it for functions like scaling. The `transform` method applies the same transformations with the previously learned features and was used on the testing dataset. This sections lists all the transformations in the order they were applied.

Minimal profile flag A binary categorical feature was added, discerning between the accounts that have numeric features and the accounts without them. This step was done first because later steps add further numeric variables, which have missing values for different users, but the intended variable was intended to capture exactly the set of users that don't have numeric features in the initial dataset.

Subscribers Count Feature While reflecting a numeric quantity, the `subscribers_count` feature was stored as a categorical variable with 1597 possi-

ble categories. To enable the machine learning algorithms, to learn from the numeric relationships, this variable was converted to be numeric.

Data Scaling To ensure equal contribution of features and faster convergence for the logistic regressor, the data was scaled with the `StandardScaler` of the `sklearn` python library.

Median imputation The `SimpleImputer` class of the `sklearn` library was used to replace all missing values for numerical features with the median of this feature. That way, it is possible for the logistic regressor to use the numeric features without creating unnecessary noise.

Categorical encoding Categorical features with exactly two categories were binary encoded, mapping each category to 0 or 1. The remaining categories were one-hot encoded, while assigning all categories with less than 30 members into an `other` bin. This `other` bin had the effect of reducing the number of city categories from 362 to only 6 and thereby preventing a drastic dimensionality increase that would have otherwise come from one-hot encoding it. For all categorical features, the `unknown` category was treated just as any other category.

3 Models

3.1 Baseline

For a baseline measurement, `sklearn`'s `DummyClassifier` was used with the "stratified" strategy. This strategy randomly predicts labels based on the frequency with which they occur in the dataset. It achieved an accuracy of 0.5 and an F1-score of 0.49 on the bot class.

3.2 Logistic Regressor

Logistic regression is one of the simplest machine learning algorithms. Despite its simplicity, it performs surprisingly well on linearly separable classification tasks. A major advantage of logistic regression is that its coefficients directly reflect feature importance for the classification task and can thus easily be interpreted. Additionally, it is very computationally efficient when dealing with large feature spaces. This makes it a good choice for local hyperparameter tuning and evaluation on the current dataset.

3.3 XGBoost

XGBoost (Extreme Gradient Boosting) is a high-performance implementation of gradient-boosted decision trees designed for speed and efficiency. ([chenXGBoostScalableTree2016](#)) It is particularly well-suited for structured datasets with a mix of categorical and numerical features, as in the current case.

One of XGBoost's primary strengths lies in its ability to model complex, non-linear relationships between features and the target variable. Unlike logistic regression, which assumes linear separability, XGBoost can capture intricate interactions through its ensemble of decision trees. It also includes built-in regularization (both L1 and L2), which helps prevent overfitting—an important consideration when working with high-dimensional data.

Another advantage is its native handling of missing values. Instead of requiring imputation, XGBoost can learn optimal default directions for missing data during tree construction, making it more robust to incomplete feature sets. Furthermore, it supports parallelized training and early stopping, allowing for efficient model tuning even on larger datasets.

4 Conclusion

Test

A Tables

Table A.1: Sorted correlations between rings and other features

	Rings
Shell weight	0.628
Height	0.610
Diameter	0.575
Length	0.557
Whole weight	0.541
Viscera weight	0.504
Shucked weight	0.421

Table A.2: Statistical comparisons of number of rings between genders

	M vs F	M vs I	F vs I
t-statistic	-3.681	27.188	29.465
p-value	<0.001	<0.001	<0.001
Cohen's d	-0.139	1.016	1.154

B Figures

Test